DA6701 – Assignment 4
Portfolio Replication

Athreya G	Kiran Kumar P	Harshvardhan Agrawal	Siddhesh Kamble
DA24B002	DA24B008	DA24B039	NA24B070

Introduction

The objective is to replicate the return profile of the S&P 500 index given for a 5 year period in the zip file, using only a subset of the stocks. Selecting k stocks out of 500 to minimise the variance is an NP-hard problem. To arrive at an approximate solution, we have used: Lasso Regression, and Latent Factor Mapping with Hierarchical Risk Parity.

Data Loading

We take X to be each ticker from the constituents, and y to be the S&P 500 stocks. We have used Jan 2020 to June 2025 (1326 days) for training the model and July 2025 to Dec 2025 (128 days) to test the model. We have removed the columns which have more than 5% missing entries, apart from other missing entries. In the end, 24 stocks were dropped due to missing entries.

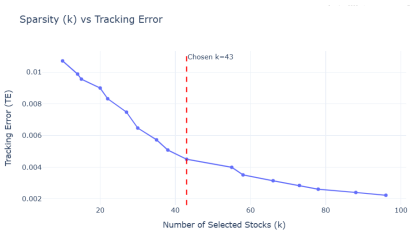
Portfolio Replication

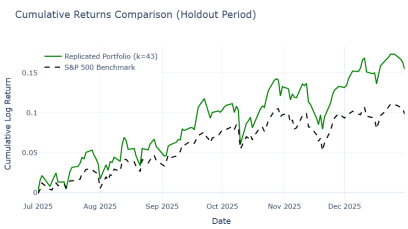
Lasso Regression

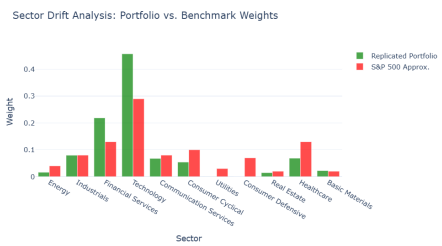
Lasso Regression is made to fit on the training data, without an intercept. We check how many stocks were given a non-zero weight and based on the max-k constraint we give, we take the k most relevant stocks to proceed, and normalise the weights accordingly.

Then we try to find the optimal alpha by generating a evenly spaced list of alpha values and building a portfolio for each alpha without any constraint on k, while keeping note of the **Tracking Error** and **Information Ratio** for each alpha on the test set. We make a dataframe on it, and ultimately choose the alpha that selects 50 or lesser constituents while minimising the tracking error.

Deliverables:







Performance Metrics:

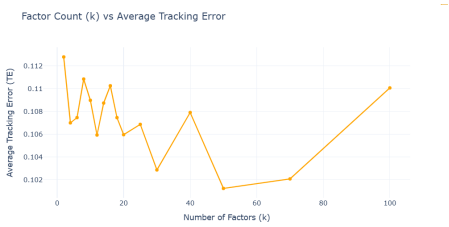
Chosen Alpha		4.5×10^{-5}
Final Count (k)		43
Tracking Error	Train	0.008204
	Test	0.004503
Information Ratio	Train	0.0536
	Test	0.0973

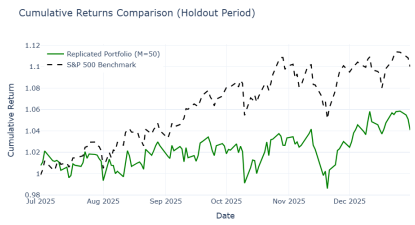
Latent Factor Mapping

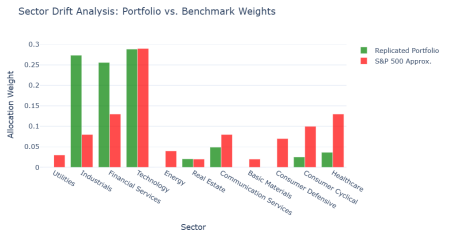
We define a neural network autoencoder with PyTorch, which takes the 500 constituents and gives k latent factors. Once the latent factors are chosen, the correlation between each stock and the latent factor is calculated, and the highly correlated stocks are taken.

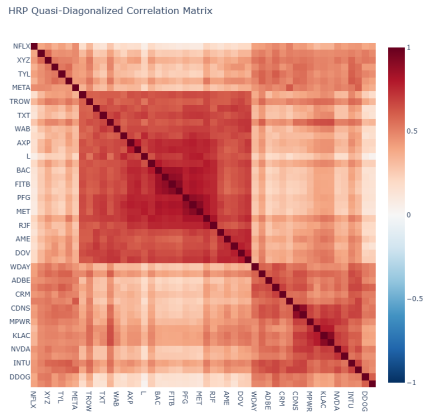
After picking these stocks, using HRP, quasi diagonalisation, and recursive binary partitioning, the weights are determined. The hyperparameters to tune are k (the number of latent factors) and M (how many total stocks to hold in the portfolio). The tuning is done and the portfolio with the least Tracking Error is taken, which was found to be **k=2, M=50, giving test TE=0.0745**.

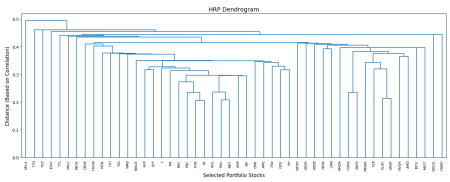
Deliverables:











Performance Metrics:
The TE and IR have been annualised (multiplied by $\sqrt{252}$).

Factors Extracted (k)		2
Final Count (M)		50
Tracking Error	Train	0.073303
	Test	0.074534
Information Ratio	Train	0.5538
	Test	-1.4328

Comparison

To compare Lasso and Latent Factor Mapping, we multiply the TE and IR of the former by $\sqrt{252}$ and then compare.

	Lasso Regression	Latent Factor Mapping
TE Train	0.130234	0.073303
TE Test	0.071482	0.074534
IR Train	0.8508	0.5538
IR Test	1.5445	-1.4328

Lasso Regression treats this like a regression problem and tries to directly predict the return price based on stock returns, while Latent Factor Mapping uses a neural network to condense the return prices to a certain number of "factors", and the correlation between stock returns and the "factors" are used to predict stock returns by making an appropriate portfolio.

Both models were able to achieve a good tracking error in the dataset, but Lasso Regression slightly outperformed Latent Factor Mapping in the test dataset.

If we look at Information Ratio, we notice that Lasso Regression has heavily outperformed Latent Factor Mapping, especially on test data. Lasso Regression could come up with a better risk-adjusted portfolio and outperformed the benchmark itself, while the Latent Factor Mapping portfolio had a severe negative value and underperformed the benchmark by a huge margin.

Ultimately, Lasso Regression outperformed Latent Factor Mapping and gave a lower tracking error while outperforming the benchmark.

Conclusion

We have explored Lasso Regression, Latent Factor Mapping, and how they were able to minimise Tracking Error with a constraint on the number of stocks chosen, an NP-hard problem approached with regularization and heuristics.